DD2497 Course Report

Mingtao Zhang (mingtaoz@kth.se)

1 THE VULNERABILITIES

Previously, the S3k project has a rudimentary file-system project, enabling applications access to FAT based file-systems. However, that implementation has the following limitations:

- **No Memory Safety:** The existing implementation is written entirely in C, featuring complex control flow and memory-intensive I/O operations without any test coverage. This creates a high risk of memory corruptions, which is a common source of high severity vulnerabilities.
- **Plaintext Storage:** The current architecture does not support disk encryption. Consequently, all data and system states are stored in plaintext, leaving them vulnerable to unauthorized access and tampering if an adversary gains physical access to the storage medium.
- **Lack of Arbitration:** There is no centralized client API or file-system server. Clients are expected to initialize their own file-system instances, which leads to race conditions and conflicted states if multiple clients attempt to update data simultaneously.

2 DESIGN CHOICES

To address the lack of memory safety in the previous C-based implementation, we chose to rewrite the userspace libraries and to implement the file-system from scratch using Rust. Rust provides strict compile-time guarantees for memory safety and data race freedom through its ownership and borrowing system, without relying on a garbage collector. This choice mitigates the risk of memory corruption vulnerabilities, such as buffer overflows and use-after-free errors, which are prevalent in low-level systems programming. Additionally, Rust's modern tooling and expressive type system facilitate the development of complex systems like a file-system server, ensuring higher reliability and maintainability.

Our design aims at "no glue code". The reason why we do not use cbindgen for the userspace library (including system calls) is mainly due to two factors. The first is that s3k's ABI highly rely on compressed data structure, that many fields are encoded in a single uint64 with non-standard bit length, which bindgen cannot generate correct code. The second is that s3k's C library is not type-safe, which means we need to write a lot of unsafe blocks in the Rust program.

When we started to draft the encryption layer, we had to choose between putting the encryption layer inside the file-system and letting it run as an extension on the block device. Filesystem level encryption has its unique advantage of having access to file-system contexts, which means the encryption settings can be customized for different files. This enables better granularity and efficiency for our encryption module, at the

expense of leaving the file metadata unprotected. On the other hand, block device level encryption protects the whole disk, including the file-system metadata and regardless of which file-system is used. However, it's an all-or-nothing approach which means the whole disk has to be encrypted which can be quite slow, and all data share a single encryption key which brings limitation to the access control. In the end, we've decided to take the block device level approach because it offers a more general file-system support and its modularity better suits the micro-kernel's philosophy.

Encryption algorithm wise, AES is the industry standard to encrypt large chunks of data efficiently. We chose to use 256 bits keys for extra security given the rise of quantum computing. Initially we were thinking about using the CBC mode of operation as it's a common choice, then it came to our attention that the CBC mode requires each 32-bytes-long block needs to be processed sequentially during the encryption process, which can be really inefficient. Additionally, there's the data expansion problem: CBC mode requires each disk sector to be encrypted with a different IV, on a physical disk, there is no extra space in a sector to store this metadata without destroying the sector alignment or reducing available capacity. Instead, we chose to use XTS mode, which supports parallelized data encryption and decryption, it also uses an implicit IV derived from the disk sector number, thus solving the data expansion problem.

In order to achieve easy-to-remember passwords for the users and resilience to brute force attack at the same time, we've decided to introduce Trusted Platform Module (TPM). A TPM is a hardware module designed to store and protect important security information, like the master disk encryption key. Instead of using the user's PIN to directly encrypt the master key, we use the PIN to unlock the TPM. The TPM maintains an internal counter of failed attempts. If an attacker attempts to brute-force the PIN, the TPM hardware will lock out after multiple failed attempts for a specified duration (exponential back-off). However, the TPM has its own limitations: its low-speed cryptographic processors are designed for infrequent cryptographic operations, not for high-speed disk I/O. This unfortunately means that we will have to release the master key used for disk encryption to the system RAM in order to have a reasonable I/O performance.

3 IMPLEMENTATION DETAILS

3.1 Filesystem Hierarchy

The file-system architecture is designed as a layered stack, ensuring modularity and separation of concerns. This design allows for easy swapping of underlying hardware drivers or file-system implementations without affecting the upper layers. From bottom to top, the hierarchy consists of:

- **Hardware Driver:** At the lowest level is the block storage device driver. In our QEMU-based environment, the driver communicates with a VirtIO block device via MMIO and manages the queues for request submission and completion.
- **Virtual Encryption Driver:** To support encryption, we implemented a virtual device driver called `VirtIOBlkDeviceWithEncryption` wrapping around the original device, which transparently encrypts data on write and decrypts on read. The encryption metadata is stored on the first sector of the physical device

while all sectors of the virtual device are offset by one when stored onto the physical device.

- **Hardware Abstraction Layer (HAL):** To decouple the file-system logic from specific hardware drivers, we defined a `BlockDevice` trait acting as a HAL. This trait exposes a simple, high-level interface with methods like `read_block`, `write_block`, `sector_size`, and `dev_size`. Any driver implementing this trait can be used by the file-system, enabling portability and easier testing. With HAL, we can implement full disk encryption without requiring any changes to the file-system code.

- **Filesystem Implementation:** We implemented a FAT32 file-system from scratch in Rust. The file-system is based on top of the `BlockDevice` trait. It encapsulates logic within the `FAT32FileSystem` struct and supports self-initialization to automatically format empty devices. The system manages cluster allocation via the FAT table and parses `FAT32Dirent` structures for directory navigation, ensuring full read/write support while adhering to the standard.

- **Virtual Filesystem Layer (VFS):** This layer provides a unified interface for file operations, abstracting away the details of specific file-systems. It defines traits such as `Inode` and `File`, allowing the system to handle different file types (e.g., regular files, directories) and potentially different file-systems uniformly. This layer handles high-level operations like path resolution before delegating to the specific file-system implementation.

- **Filesystem Server:** The file-system server is a service acting entirely within the boundary of a regular user-space application. It handles all file related calls from other applications through the socket capability of S3k kernel and manages the encryption state of disks. In our S3k specific implementation, it also handles the initialization and capability provisioning for client applications for demonstration purposes.

3.2 Encryption Key Management with TPM

In our project, we implemented a dedicated TPM 2.0 driver that is capable of generating, sealing and unsealing the master encryption key for the system.

The driver was implemented according to the Trusted Interface Specification (TIS) standard. The TPM registers are mapped into memory at a specific registers and the driver works by reading and writing to these addresses. The most important registers are:

- **Access register:** Controls access to the TPM and is used to request and confirm ownership of the TPM before sending commands.

- **Status register:** Shows the current state of the TPM, such as whether it is ready to accept a command or if data is available to read.

- **FIFO buffer:** The buffer that is used to write commands to the TPM and read responses from the TPM.

Before any command can be sent to the TPM the driver has to access locality 0 which is the base access level needed for our use case. Once the TPM is ready the command is written into the FIFO buffer one byte at a time, the driver tells the TPM to process the command, then the tpm signals that it is finished and the response can be read from the FIFO.

The driver supports several basic TPM functions like generating random values for keys and TPM startup during system initialization.

For secure key storage the driver also generates the primary root key inside the TPM. This key can never be extracted and is used as a secure parent for other keys. A master encryption key is then locked inside the TPM and protected with the user pin. The TPM returns a public blob and a private blob. These blobs are safe to store on disk, but they cannot be used to recover the key without the TPM and the correct PIN.

When the system needs to use the master key, the sealed blobs are loaded back into the TPM and unsealed using the correct PIN. If the PIN is incorrect, the TPM will reject the request. After too many failed attempts, the TPM will enter a lockout state to prevent attacks. This adds an extra layer of protection for the stored encryption key and improves the overall security of the system.

4 MITIGATION OF VULNERABILITIES

Our implementation counter the previously mentioned vulnerabilities in the following ways:

- **Memory Safety Guarantees:** By transitioning the implementation to Rust, we have eliminated the class of memory corruption vulnerabilities like buffer overflows, dangling pointers, and data races which are inherent in the legacy C codebase. Rust's affine type system and ownership model enforce memory safety at compile-time, without the runtime overhead of garbage collection.

- **Data Confidentiality:** The introduction of the block device encryption layer ensures that data on disk is protected by AES-256-XTS, rendering the storage unreadable to attackers with physical access. Furthermore, the TPM integration effectively mitigates brute-force attacks against the encryption key. By offloading the PIN verification to the TPM hardware, we enforce an exponential back-off policy that makes such attacks infeasible, even for simple user passwords.

- **Serialized File Operations:** The implementation of the centralized file-system server resolves the arbitration issues. By restricting direct block device access and forcing all client applications to communicate via IPC, the server acts as a single source of file operations. It serializes concurrent requests, thereby eliminating race conditions and preventing the corruption of data when multiple uncoordinated clients write to the disk simultaneously.

5 INDIVIDUAL CONTRIBUTION

The whole Rust version of the s3k userspace library and the filesystem hierarchy below Filesystem Server are fully completed by me. The userspace library is a replacement of the `alt_c` library including syscall asm, capability related functions and a memory allocator. The filesystem parts include MMIO driver, HAL, FAT32 implementation, and VFS interfaces.

```
/mnt/d/Documents/KTH/DD2497/s3k  2024-sys-sec +1 !1  root@Nullptr-Laptop  16:47:48
git log --author="Mingtao" --pretty=tformat: --numstat | awk '{ add += $1; subs += $2; loc += $1 - $2 } END { printf
"Added lines: %s, Removed lines: %s, Total lines: %s\n", add, subs, loc }'
Added lines: 7230, Removed lines: 329, Total lines: 6901
```

Figure 1. Git contributions

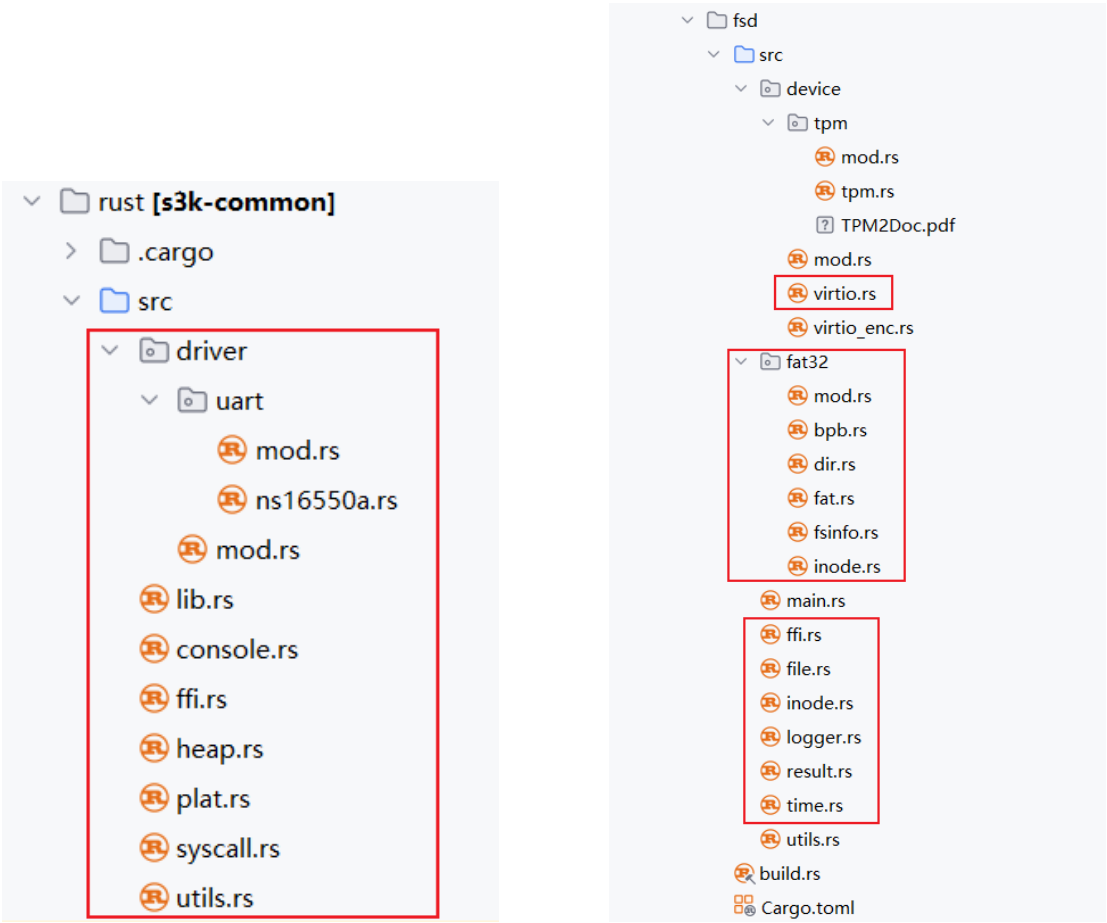

Figure 2. File contributions